B-Wings · Manual de reingeniería, investigación y especificación técnica

Reconstrucción fiel de la versión Famicom/NES de 1986 — Documento de referencia / tesis técnica

Oct 9, 2026 · Versión 0.3 · Edición unificada · Investigación en curso

> **Objetivo del documento.** Documentar el juego de forma comprensible para cualquier lector y establecer una base verificable para una futura reconstrucción independiente. Este manual es la «biblia» del proyecto: reúne cada detalle del funcionamiento de B-Wings junto con su nivel de evidencia, de modo que un programador pueda reconstruir el comportamiento en el lenguaje o motor que prefiera.

Ficha del documento

Campo	Valor
Título	B-Wings — Manual de reingeniería, investigación y especificación técnica
Juego estudiado	B-Wings (Data East), versión Famicom/NES de 1986
Versión del manual	0.3 — edición unificada
Fuentes unificadas	Manual v0.2 (investigación), Guía técnica de arquitectura NES y reimplementación limpia en Godot, y los últimos hallazgos sobre WINGs, Morph y Laser Wall
Meta del proyecto	Ejecutable de escritorio con tres modos: Original, Corregido/Enhanced y Rebuild extendido
Motor previsto	Godot 4 (diseño de referencia); la elección definitiva se confirma al cerrar la especificación
Estado de la investigación	En curso — ver el mapa de conocimiento (capítulo 25)
Idioma	Español

Cómo leer este manual

Cada afirmación lleva implícita una clasificación de evidencia (HECHO, INFERENCIA, HIPÓTESIS, DECISIÓN o PENDIENTE; capítulo 3). Las direcciones de memoria se escriben con el prefijo `$` y entre comillas de código, por ejemplo `$00CF`. Los recuadros con sangría resaltan reglas, principios y conclusiones. Las secciones marcadas como pendientes son huecos reales de la investigación, no omisiones del texto.

Cada vez que el equipo o su asistente de investigación descubra algo nuevo, se incorpora al capítulo correspondiente y se actualiza el mapa de conocimiento y el registro de cambios del Apéndice C.

Índice

Parte I · Fundamentos

1. Propósito y filosofía
2. Alcance y criterios legales
3. Método de investigación

Parte II · Hardware y cartucho

4. Arquitectura de hardware NES
5. Identificación del juego y de la ROM

Parte III · El juego

6. Modelo de gameplay
7. Sistema de WINGs
8. Sistema Morph
9. Estructuras
10. Enemigos
11. Etapas
12. Bosses

Parte IV · Ingeniería inversa de la ROM

13. Arquitectura interna observada
14. Mapa de RAM
15. Scrolling
16. Proyectiles y entidades
17. Slowdown
18. Glitches y colisiones anómalas

19. Agenda de investigación en la ROM

Parte V · Reimplementación en PC

20. Qué conservar y qué mejorar
21. Arquitectura propuesta
22. Validación y pruebas de regresión
23. Plan de desarrollo
24. Documentación, distribución y próximos pasos

Parte VI · Estado y cierre

25. Mapa de conocimiento
26. Conclusión

Apéndices

A. Glosario · B. Fuentes y referencias · C. Reconciliación y registro de cambios · D. Plantillas de registro

Parte I · Fundamentos

1. Propósito y filosofía

El proyecto no busca hacer un remake moderno ni un simple port. La pregunta central es: ¿cómo sería este mismo B-Wings si se mantuvieran sus reglas, niveles, armas, enemigos, dificultad, estética y ritmo, pero se eliminaran las limitaciones técnicas que no forman parte del diseño?

De esa pregunta salen dos productos complementarios: un **manual** que documenta cada comportamiento con evidencia (este documento) y un **motor** para PC que lo reproduce y, opcionalmente, lo amplía.

Orden de prioridad

1. Fidelidad al original.
2. Corrección de limitaciones técnicas demostradas.
3. Mejoras opcionales.
4. Arquitectura simple para iterar rápidamente.

1.1 Fidelidad

Fidelidad significa reproducir no sólo gráficos, sino también movimiento, aceleración, cadencia de disparo, límites de proyectiles, estados de daño, comportamiento enemigo, scrolling, colisiones, progresión y timing.

1.2 Modos de juego del ejecutable

En lugar de mantener «versiones» separadas del programa, el ejecutable usa perfiles de parámetros cargados desde archivos de datos y activados por configuración.

Modo	Objetivo	Qué cambia
Original	Reproducir el comportamiento medido del Famicom.	Nada: reglas, límites, slowdown y glitches confirmados se conservan.
Corregido / Enhanced	Conservar las reglas y eliminar limitaciones técnicas demostradas.	Correcciones de glitches, estabilidad de timing (por ejemplo 60 FPS nativos), ausencia de límite artificial de sprites. Las correcciones se activan sólo de forma explícita.
Rebuild / Extendido	Plataforma para contenido nuevo sin perder la base del original.	Más niveles, más enemigos, más disparos en pantalla y niveles creados por la comunidad mediante datos.

> **Regla.** Enhanced no significa hacerlo más fácil. Una mejora técnica no debe cambiar las reglas de diseño. Los cambios de contenido pertenecen exclusivamente al modo Rebuild.

> **Decisión pendiente de confirmar.** El manual v0.2 sólo definía los modos Original y Enhanced; la guía técnica proponía «original, corregido, extendido». Esta edición los unifica en tres modos. Si el proyecto prefiere fusionar Corregido con Enhanced, el contenido de la tabla se mantiene.

2. Alcance y criterios legales

Este manual describe cómo analizar hardware y software de la NES con fines de estudio, interoperabilidad y diseño de motores propios. No contiene código desensamblado, tablas de datos extraídas de juegos concretos, gráficos, música ni mapas de niveles de ningún título comercial. Registra direcciones de memoria, comportamientos observados y especificaciones propias.

2.1 Principios de trabajo

- **Reimplementación limpia (clean room).** Quien documenta el comportamiento no debe ser quien escribe el código final, o al menos debe trabajar sólo con especificaciones de hardware y observaciones propias.
- **Assets propios.** Todo sprite, sonido y nivel de la versión distribuida debe ser creado desde cero o contar con licencia explícita.
- **ROM legal.** Cualquier prueba con la ROM original debe hacerse con una copia que el usuario posea legalmente, sin distribuirla.

2.2 Estrategia clean-room

La estrategia recomendada es la reconstrucción independiente: documentar comportamiento, estados y tablas, y escribir una implementación nueva que produzca esos resultados. No se debe trasladar código propietario de la ROM.

> **Consecuencia.** La protección del cartucho se estudia para entender la ROM, pero no se copia como gameplay en PC. Lo mismo vale para cualquier rutina interna: se documenta qué hace, no cómo está escrita.

3. Método de investigación

Toda afirmación del manual se clasifica en una de cinco categorías de evidencia. Esto evita convertir suposiciones en hechos y permite saber en cualquier momento qué parte de la especificación está firme y cuál no.

3.1 Clasificación de las afirmaciones

Categoría	Significado	Ejemplo en este manual
HECHO / OBSERVADO	Comprobado en la ROM o en emulación, o corroborado por fuentes independientes.	`$00CF` funciona como selector de WING.
INFERENCIA	Conclusión razonable derivada de hechos, pero no observada directamente.	`$00A0/$00A1` participan en el estado que permite el glitch del Laser Wall.
HIPÓTESIS	Explicación posible que todavía debe ponerse a prueba.	Las colisiones son la causa principal del slowdown.
DECISIÓN	Elección de diseño del proyecto, no un descubrimiento.	Enhanced debe eliminar una limitación técnica comprobada.

Categoría	Significado	Ejemplo en este manual
PENDIENTE	Aún no investigado o sin evidencia suficiente.	Causa exacta del glitch del Laser Wall.

Además de la categoría, el mapa de RAM y el mapa de conocimiento usan un nivel de confianza: **Alto**, **Medio/Alto**, **Medio**, **Bajo/Medio**, **Bajo** o **Pendiente**.

3.2 Cadena de análisis

Cada problema detectado recorre la misma cadena, sin saltarse pasos:

1. Problema.
2. Causa probable.
3. Evidencia.
4. Comportamiento observable.
5. Clasificación (según la tabla anterior).
6. Solución.
7. Impacto sobre el gameplay.
8. Prueba.

> **Principio.** «Se siente lento» no es una causa. Hay que medir timing, carga, objetos, colisiones y tareas del motor antes de afirmar por qué algo ocurre.

3.3 Procedimiento de observación

El estudio del juego sigue cinco pasos. Documentan comportamiento, no código.

1. **Identificación del cartucho.** Leer el encabezado iNES (16 bytes) para obtener mapper, tamaño de PRG/CHR y mirroring. Herramientas: editores hexadecimales y bases de datos de cabeceras.
2. **Observación en emulador con depuración.** Usar un emulador con visor de memoria, breakpoints y trazas para registrar qué direcciones cambian en cada evento del juego (pausa, disparo, colisión).
3. **Mapeo de estados.** Documentar por escrito la máquina de estados observada (título, juego, pausa, game over) con transiciones y disparadores. Esto es comportamiento, no código.
4. **Medición de tiempos.** Contar frames entre eventos para estimar velocidades, cadencias de disparo y tiempos de invulnerabilidad. Registrar los resultados en tablas propias.

5. **Verificación independiente.** Cada hipótesis debe comprobarse con al menos dos observaciones reproducibles antes de incluirla en la documentación como hecho.

3.4 Plantilla de registro

Ejemplo ficticio del formato con el que se registra cada hallazgo (la plantilla completa está en el Apéndice D):

ID	Evento observado	Frames medidos	Hipótesis	Verificado
E-01	Cadencia de disparo	8	Temporizador de 8 frames	Sí (3 pruebas)
E-02	Pausa	—	Bandera en RAM de estado	Pendiente

3.5 Limitación, bug o decisión de diseño

Antes de tocar cualquier comportamiento hay que separar tres conceptos que a menudo se confunden:

Concepto	Definición	Ejemplo
Limitación	Algo que el hardware o la arquitectura impone.	Cantidad máxima de entidades simultáneas (hipotético).
Bug	Una implementación que produce un comportamiento incorrecto.	Glitch del Laser Wall.
Decisión de diseño	Algo que parece una limitación pero puede haber sido deliberado.	Limitar la cantidad de proyectiles para controlar la dificultad.

> **Regla.** No se corrige nada hasta saber a cuál de las tres categorías corresponde. Una limitación puede desaparecer en PC; un bug puede corregirse en el modo Enhanced; una decisión de diseño se conserva.

Parte II · Hardware y cartucho

4. Arquitectura de hardware NES

Para reimplementar B-Wings hay que distinguir qué comportamientos nacen del diseño del juego y cuáles del hardware que lo ejecutaba. Este capítulo resume el hardware relevante; las referencias públicas están en el Apéndice B.

4.1 Componentes

Componente	Función	Detalle relevante para reimplementar
CPU Ricoh 2A03	Núcleo 6502 sin modo decimal	~1,79 MHz (NTSC); 2 KB de RAM interna
PPU 2C02	Gráficos	256×240 px, 64 colores de paleta, 8 sprites por línea
APU	Sonido	2 pulsos, triángulo, ruido, canal DMC
Cartucho / mapper	Bancos de PRG y CHR	Determina cuánta ROM y VRAM se pueden direccionar
Controladores	Entrada	Registros `$4016/$4017`, lectura serial de 8 botones

4.2 Mapa de memoria de la CPU (resumen)

Rango	Contenido
`$0000–$07FF`	RAM interna (espejada hasta `$1FFF`)
`$2000–$2007`	Registros de la PPU (espejados hasta `$3FFF`)
`$4000–$4017`	APU, DMA de sprites (`$4014`) y controladores
`$4020–$FFFF`	Espacio de cartucho (PRG-RAM, PRG-ROM, vectores)

4.3 Por qué importa para el proyecto

Con unos 2 KB de RAM y un presupuesto de CPU fijo por cuadro, cualquier juego de NES reparte su trabajo en tareas que deben terminar antes del siguiente cuadro; cuando no terminan aparecen el slowdown y la pérdida de sincronía (capítulo 17). La PPU, además,

sólo puede mostrar 8 sprites por línea, lo que explica el parpadeo típico de la plataforma. Estas son **limitaciones de hardware** en el sentido de la sección 3.5: en PC pueden desaparecer, pero sólo después de comprobar que no forman parte del diseño del juego.

5. Identificación del juego y de la ROM

B-Wings fue desarrollado por Data East. La versión Famicom de 1986 contiene 30 niveles y adapta el arcade de 1984 [1].

Dato	Valor
Archivo	B-Wings (J).nes
PRG-ROM	32 KiB
CHR-ROM	8 KiB
PRG CRC32	725D5CC4
CHR CRC32	E4075FC0
Mapper	185
Mirroring	Horizontal

5.1 El mapper 185

El mapper 185 utiliza lógica de protección que puede deshabilitar selectivamente la CHR-ROM; por eso algunas escrituras del arranque son comunicación con el cartucho y no variables normales del juego [3].

> **Consecuencia.** La protección se estudia para entender la ROM, pero no se copia como gameplay en PC.

Parte III · El juego

6. Modelo de gameplay

El jugador controla el FX-1 en un shooter vertical y obtiene módulos WING que cambian radicalmente su capacidad ofensiva. La versión Famicom tiene 30 etapas [1][F-A].

6.1 Dos alturas de juego

La documentación describe dos alturas: sin WING se puede descender a un nivel inferior, donde ciertos enemigos aéreos dejan de afectar al jugador, aunque siguen existiendo amenazas terrestres y obstáculos [F-A].

6.2 Scroll lateral cíclico

La pantalla continúa lateralmente de forma cíclica; por tanto, el movimiento horizontal no debe modelarse como un simple rectángulo con dos paredes [F-A].

6.3 Selección inicial de WING

La selección inicial de WING aparece en las etapas 1, 5, 9, 13, 17, 21, 25 y 29. El patrón 4n+1 está documentado [F-B].

Etapa con selección	1	5	9	13	17	21	25	29
n (en 4n+1)	0	1	2	3	4	5	6	7

6.4 Estados de WING

La pérdida de un WING no equivale necesariamente a la pérdida inmediata de una vida. El sistema posee estados diferenciados de WING equipado, WING perdido, destrucción y recuperación. Las transiciones exactas deben verificarse directamente antes de fijarlas (PENDIENTE).

7. Sistema de WINGs

Los WINGs son sistemas de comportamiento, no simples cambios de sprite. Alteran cantidad, dirección, geometría, penetración, defensa y reglas de colisión. Por eso el manual los trata como un sistema con capas y no como una lista de nombres de armas.

7.1 Catálogo de WINGs

WING	Función / comportamiento a verificar
FX-1	Disparo básico; límite y cadencia deben medirse.
Cannon	Tres proyectiles en rápida sucesión.
Wide	Disparo abierto; regla especial al impactar.
Multi	Proyectiles con diferentes ángulos.
Van	Barrera de proyectiles de alcance corto.

WING	Función / comportamiento a verificar
Side	Ataques laterales y frontal.
Anti	Ataques hacia delante y atrás.
Hammer	Disparo + elementos orbitales defensivos.
Jump	Proyectiles que saltan obstáculos.
Dyna	Rayo corto con capacidad de destrucción continua.
Fire	Llamas capaces de penetrar enemigos y bosses.
Ocultas	Ring Laser, Multi-Hammer y X-Ray (según el manual v0.2); comportamiento exacto pendiente de medición. Ver la reconciliación con los nombres del selector en la sección 7.3 y en el Apéndice C.

Las funciones generales de estas WINGs están corroboradas por documentación de gameplay [2][4].

7.2 El selector `$00CF`

La evidencia independiente confirma que `$00CF` funciona como selector de WING. Está documentado directamente por Whipon y aparece también en bases independientes de códigos. Clasificación: **HECHO**, confianza Alta.

`$00CF`	WING
`$01`	Cannon
`$02`	Wide
`$03`	Multi
`$04`	Van
`$05`	Side
`$06`	Anti
`$07`	Hammer
`$08`	Jump
`$09`	Dyna
`$0A`	Fire
`$0B`	Hammer Laser

`$00CF`	WING
`$0C`	4 Way Laser Wall
`$0D`	Multi Hammer

Las fuentes consultadas no documentan el valor `$00`. No se asume su significado (por ejemplo, «sin WING» o FX-1 básico): queda PENDIENTE.

7.3 Una capa de armas especiales sobre el selector común

Lo realmente interesante es lo que aparece alrededor del selector. Los códigos publicados documentan:

- Start With Hidden Weapon 1 y Start With Hidden Weapon 2.
- Enable Secret Weapons Morphs.
- Enable Unlimited Morphs.
- Can Morph Always.
- Don't Lose Your Weapon After Crashing.

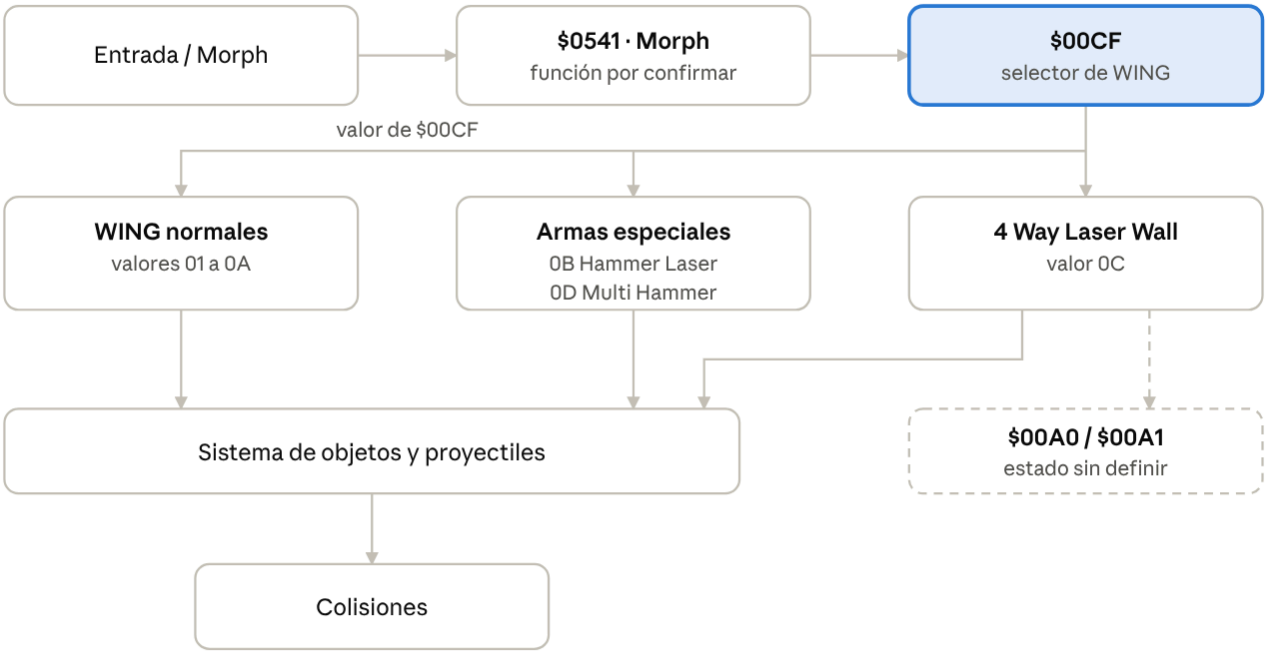
Una guía de códigos especifica qué hacen las dos armas ocultas:

Arma oculta	Comportamiento descrito	Valor asociado
Hidden Weapon 1	Produce grandes burbujas y tiene dos bolas que protegen al jugador como escudo.	…01/0B → `$00CF` = `$0B`
Hidden Weapon 2	Dispara ocho bolas en diferentes direcciones.	…01/0D → `$00CF` = `$0D`

Otra fuente que traduce los códigos internos del emulador muestra que los códigos de inicio para las armas ocultas modifican valores terminados en 0B y 0D, lo que coincide exactamente con `$00CF` = `$0B` y `$00CF` = `$0D`. Esto permite establecer con bastante confianza que **0B y 0D no son valores arbitrarios de un cheat, sino estados funcionales del sistema de armas**.

> **Precisión importante.** No debe llamarse a 0B, 0C y 0D simplemente «WINGs normales». El juego parece tener una capa de armas especiales o secretas encima del selector común. La relación exacta entre estos tres valores y los nombres Ring Laser, Multi-Hammer y X-Ray del manual v0.2 está pendiente (Apéndice C).

Un selector, tres familias de armas: modelo de investigación



Línea continua: flujo hipotético. Línea discontinua: relación funcional sin definir.
Es un modelo de investigación, no una descripción definitiva del código.

arquitectura probable del sistema de armas · hipótesis de investigación

El diagrama resume la arquitectura probable del sistema de armas: el selector `$00CF` reparte el control entre WINGs normales (01–0A), armas especiales (0B y 0D) y el Laser Wall (0C), y las tres familias terminan en el sistema de objetos y proyectiles. Es un modelo de investigación, mucho más fuerte que el anterior, pero todavía no una descripción definitiva del código.

7.4 Códigos publicados y qué revelan

Código publicado	Dirección / valor	Qué sugiere	Clasificación
Start With Hidden Weapon 1 / 2	Valores …01/0B y …01/0D	`$00CF` = `$0B` y `$0D` son armas reales del juego	HECHO
Enable Secret Weapons Morphs	No registrada	Las armas secretas pueden usarse durante Morph	PENDIENTE
Enable Unlimited Morphs / Can Morph Always	No registrada	La cantidad de Morph y la posibilidad de hacerlo son conceptos distintos	INFERENCIA

Código publicado	Dirección / valor	Qué sugiere	Clasificación
Infinite Morphs	`$0541` = `$09`	Posible recurso o estado de Morph (ver capítulo 8)	INFERENCIA
Don't Lose Your Weapon After Crashing	No registrada	Por defecto el arma se pierde al chocar: existe un estado de «crash»	INFERENCIA
Increase Van Range / Increase Dyna Range	No registrada	El alcance de las armas es un parámetro modificable	INFERENCIA
Disable 4 Way Laser Wall Glitch	`$00A0` = `$FF`, `$00A1` = `$FF`	Estado asociado al glitch (ver sección 18.1)	HECHO (relación) / PENDIENTE (causa)

7.5 Parámetros por WING que deben reconstruirse

Los códigos Increase Van Range e Increase Dyna Range muestran que el comportamiento de las armas no está necesariamente codificado como una propiedad indivisible. Probablemente existan parámetros o tablas para alcance, velocidad, cantidad, dirección, duración, tipo de proyectil y patrón de disparo (HIPÓTESIS).

Esto abre una línea de investigación concreta: para cada WING hay que reconstruir una ficha completa, mucho más útil para la implementación que una lista de nombres.

Campo de la ficha de WING	Qué debe registrar
Código / ID	Valor de `$00CF` y nombre
Proyectil	Tipo y apariencia lógica del objeto creado
Cantidad simultánea	Proyectiles o elementos activos a la vez
Velocidad	Desplazamiento por tick
Dirección	Ángulos o patrón de salida
Alcance	Distancia o duración máxima de vuelo
Duración	Tiempo de vida de los objetos persistentes
Cadencia	Frames entre disparos
Comportamiento al impactar	Qué ocurre al golpear enemigos
Interacción con estructuras	Puntos débiles, sectores y reglas especiales

Campo de la ficha de WING	Qué debe registrar
Interacción con bosses	Daño, penetración y fases
Después de Morph / crash	Estado del arma tras transformarse o chocar

7.6 Los WINGs como datos

> **Arquitectura.** Cada WING debe representarse como datos: patrón de disparo + proyectiles + colisión + defensa + reglas especiales + visual. El motor interpreta esos datos; no hay una clase de código por arma.

8. Sistema Morph

`$0541` probablemente pertenece al sistema Morph, pero todavía no debe llamarse «contador». Hay un código publicado que lo fija en `$09` (Infinite Morphs), y además existen códigos separados: Enable Unlimited Morphs, Enable Secret Weapons Morphs y Can Morph Always. Eso indica que hay al menos tres conceptos distintos detrás de la palabra «Morph».

8.1 Tres conceptos que no deben mezclarse

Concepto	Pregunta que debe responder la investigación	Código publicado relacionado
Cantidad / recurso	¿Cuántos Morph quedan y cómo se gastan?	Infinite Morphs (`$0541` = `$09`), Enable Unlimited Morphs
Posibilidad de hacer Morph	¿Qué condiciones habilitan la transformación?	Can Morph Always
Armas permitidas durante Morph	¿Qué valores de `$00CF` pueden usarse transformado?	Enable Secret Weapons Morphs

> **Advertencia para la implementación.** Sería un error escribir `int morphs = 9;` en el PC y dar por cerrado el análisis. Primero hay que descubrir qué hace realmente `$0541` .

8.2 Qué puede ser `$0541`

Candidatos a verificar: contador, estado, máscara, cantidad de Morph, disponibilidad o una combinación de funciones según el contexto. El manual v0.2 lo clasificaba como «estado morph/WING» con confianza Alta; esta edición lo baja a **Medio/Alto** como parte del Morph y **sin función definida**, hasta resolver los accesos descritos en la sección 19.2.

9. Estructuras

Las estructuras no son simplemente enemigos grandes. Muchas tienen sectores, puntos débiles, direcciones de ataque o interacciones específicas.

Familias documentadas	Tratamiento recomendado
H, Octagon, Tap, Cable	Entidades compuestas con partes y puntos vulnerables.
Side, Neon, Sputnik	Múltiples segmentos o vulnerabilidades direccionales.
Land Mine	Colisión amplia.
Solid	Regla especial asociada a Jump.
Delta	Barricada con punto protegido.
Mix & Match	Composición de estructuras.

> **Conclusión.** La colisión debe ser un sistema de reglas, no una sola caja por sprite.

10. Enemigos

La documentación histórica enumera numerosas familias: Bombs, Bats, Sirens, Octopi, Diamonds, Syringes, Cabbages, Barrels, Homing Drills, Darts, Spiked Cymbals, Cupcakes, Toy Aeroplanes, Frying Pans, Retractors, Stern Bats, Space Shuttles, Caterpillars, Carrots y Y-Flyers.

Cada enemigo deberá modelarse mediante estados: spawn, entrada, movimiento, ataque, vulnerabilidad, retirada, muerte y eliminación.

10.1 Datos a medir por enemigo

- Posición de aparición.
- Momento de activación.
- Trayectoria.

- Velocidad y aceleración.
- Seguimiento al jugador.
- Cadencia y cantidad de disparos.
- Hitbox.
- Duración de estados.
- Condición de retirada.
- Puntuación.

11. Etapas

Las 30 etapas deben convertirse en datos: fondo, perfil de scroll, secuencia de estructuras, enemigos, objetos, WING y boss. El motor interpreta esos datos; no hay lógica de nivel escrita a mano.

La existencia de 30 etapas está corroborada por fuentes independientes; también existe un playthrough completo útil para comparación visual [1][5]. Las observaciones históricas sobre mayor cantidad de enemigos y proyectiles en etapas avanzadas son una pista para estudiar el slowdown, no una prueba de causalidad (capítulo 17).

Campo de datos por etapa	Contenido
Fondo	Gráficos de fondo y perfil visual
Perfil de scroll	Velocidad y formato de desplazamiento
Secuencia de estructuras	Orden y posición
Enemigos y objetos	Tipos, tiempos de aparición, trayectorias
WING	Selección o entrega de WING (etapas 4n+1 para la selección)
Boss	Tipo, variante de etapa y reglas

12. Bosses

Los Gobunasu aparecen al final de las etapas. No se debe asumir que cada jefe tiene una vida fija independiente de la etapa. El modelo previsto es:

> **Modelo de boss.** tipo de boss + variante de etapa + reglas de impacto + vulnerabilidad + fases + proyectiles + condición de derrota.

PENDIENTE. Registrar golpes, WING utilizado, puntos vulnerables, fases y proyectiles de cada jefe.